IST 387: Introduction to Applied Data Science
Lab 8: Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

- Supervised and Unsupervised Machine Learning
- Linear Modeling
- An Example- Car Maintenance
- Using the Caret Package
- Partitioning into Training and Cross Validation Datasets
- Using k-Fold Cross Validation
- Important R functions

SUPERVISED AND UNSUPERVISED MACHINE LEARNING

Machine learning (ML) refers to a variety of computer algorithms that process data and find patterns. ML falls into two broad categories:

- **Supervised learning technique:** Supervised ML requires labelled input and output data during the training phase of the ML model lifecycle. This training data is often labelled by a data scientist in the preparation phase, before being used to train and test the model. Once the model has learned the relationship between the input and output data, it can be used to classify new and unseen datasets and predict outcomes. The reason it is called supervised ML is because human interaction is required to accurately label data for supervised learning.

- **Unsupervised learning technique:** Unsupervised ML is the training of models on raw and unlabeled training data. It is often used to identify patterns and trends in raw datasets, or to cluster similar data into a specific number of groups. These algorithms discover hidden patterns in data without the need for human intervention (hence, they are "unsupervised").

Now, let's explore the difference between classification and regression in ML:

Classification is the process of finding or discovering a model or function that helps in separating the data into multiple categorical classes. In classification, data is categorized under different labels according to some parameters given in the input and then the labels are predicted for the data.

Regression is the process of finding a model or function for distinguishing the data into continuous real values instead of using classes or discrete values. It can also identify the distribution movement depending on the historical data.

A very simple algorithm for prediction is **linear regression**. A relatively straightforward application of mathematics allows linear regression to transform a dataset into a linear equation.

For example: We could use the number of oil changes for a vehicle to predict the repair costs for that vehicle. All we would need is some previous data on some vehicles' oil changes and repair costs.

Linear regression would use these data to calculate a slope to go with the number of oil changes and an intercept. Then we could plug in a new value for the number of oil changes to learn the predicted cost of repairs.

We could use a ML model to predict the weather in future days:

1. If we were predicting rain/no rain, we would use a classifier, and
2. If we were predicting the amount of rain that would be regression.

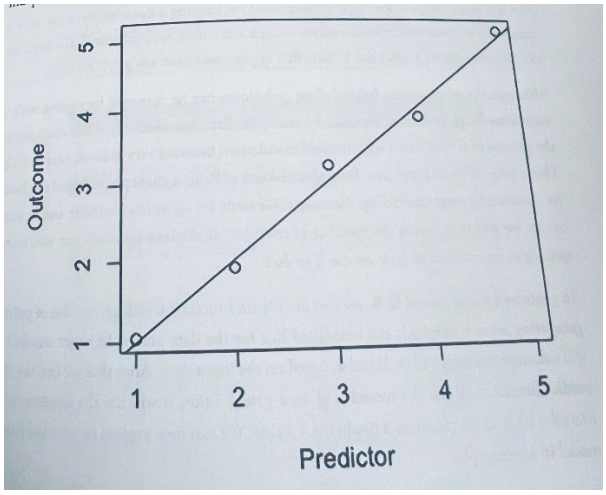
LINEAR MODELING

Finding relationships between variables is one of the basic aims of data science. The question, "Does X influence Y?" is of prime concern for data analysts from every sector: Are house prices

influenced by incomes? Is the growth rate of crops improved by fertilizer? Do taller sprinters run faster? We can answer these kinds of questions by building predictive models.

One of the simplest forms of predictive model is called a **linear model** because it uses the slope and the intercept of a line to represent the relationship between an X (predictor) variable and a Y (outcome) variable. It is easy to visualize how this works with one variable changing in step with another variable. This method is often used by statisticians and is known as linear modeling.

The following chart shows a line fitted to a series of points, using the **least squares method** (a relatively simple method of finding a best-fitting line). The chart shows how the relationship between an input (independent) variable on the horizontal X-axis relates to the output (dependent) values on the Y-axis.



Note that although the line fits the points well, with an even split of points above and below the line, none of the points are precisely on the line—the training data do not fit the line precisely. The mathematical idea that allows us to create lines of best fit for a set of data points like this is that we can find a line that will minimize the squared distances between the line and each of the points. Once the model calculates the line that minimizes the squared distance for all the points, the model represents the line with an equation you might remember from algebra:

$$\boxed{Y=bX+a.}$$

Do not worry if you have never taken an algebra course: all you need to understand is that the equation has two values that describe the line (b and a). The letter "b" denotes the slope of the line and the letter "a" describes where the line crosses the Y-axis, also known as the V-intercept.

These values are calculated by applying some math to the training data. In practice, most datasets used for training a linear model have dozens, hundreds, or thousands of observations. Once b and a have been calculated, the equation can then be used to predict the Y value that might result from any given X value. Although the mathematics behind these techniques can be managed by anyone with a little knowledge of matrix algebra, the reality is that even with only a few data points, the process of fitting a line with manual calculations becomes very tedious, very quickly. This is why

software programs, from spreadsheets to R, all include prepackaged methods for conducting linear modeling. Because these tools are so widely available and so easy to use, we will not discuss the specifics of how the calculations are done but will move quickly to an example of how we use R to do it.

To generate a linear model in R:

- We can use the **lm** function, which given a list of points, generates the best-fitted line for these data points. In other words, **lm** will calculate the appropriate b and a, based on the input data.

- After that we can use the **predict** function: When the model is given a new X value, it will use the equation of a line (Y = bX+ a), to calculate a predicted Y value.

AN EXAMPLE-CAR MAINTENANCE

Is changing the oil of a car more frequently a good thing? Does it save money in the long run? Let's use linear regression to address this question.

Let's say that we were just put in charge of maintaining a fleet of company cars. We know that the company replaces the cars every three years, and in fact the company just replaced the cars (i.e., bought a new fleet of cars). The person who was previously in charge of the car maintenance didn't have a schedule for when to change the oil, but rather changed the oil whenever the car happened to be available. Luckily, good maintenance records were kept, and now we want to try and figure out, in a data-driven approach, if changing the oil frequently is a good idea. The following R code defines our dataset:

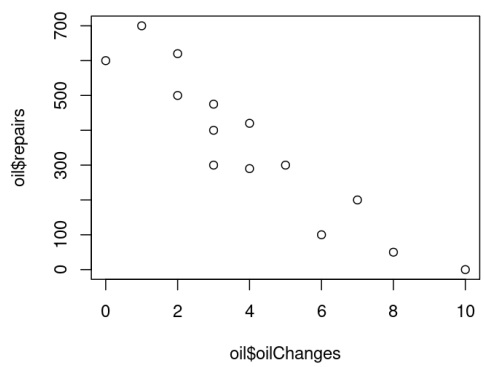
```
oilChanges <- c(3,5,2,3,1,4,6,4,3,2,0,10,7,8)
repairs <- c(300, 300, 500, 400, 700, 420, 100,
             290, 475, 620, 600, 0, 200, 50)
miles <- c(20100, 23200, 19200, 22100, 18400, 23400, 17900,
           19900, 20100, 24100, 18200, 19600, 20800, 19700)
oil <- data.frame(oilChanges, repairs, miles)
View(oil)
```

	oilChanges	repairs	miles
1	3	300	20100
2	5	300	23200
3	2	500	19200
4	3	400	22100
5	1	700	18400
6	4	420	23400
7	6	100	17900
8	4	290	19900
9	3	475	20100
10	2	620	24100
11	0	600	18200
12	10	0	19600
13	7	200	20800
14	8	50	19700

Let's look at the data for the cars that were just replaced. The dataset provides three columns of information. First, there is the number of **oil changes** that each car had during the past three years. The next column shows the total amount of **repairs** (in dollars. Finally, we have the **miles driven** by each car.

Given these data, can we determine whether changing the oil frequently is a good thing. Our independent variables are **oilChanges and miles**. The dependent variable (the one we are trying to predict) is **repair costs**. Before building a model, we should do some exploratory analysis, which professional data scientists always do prior to building a model. In this case we can plot the points with the following R command:

```
plot(oil$oilChanges, oil$repairs)
```

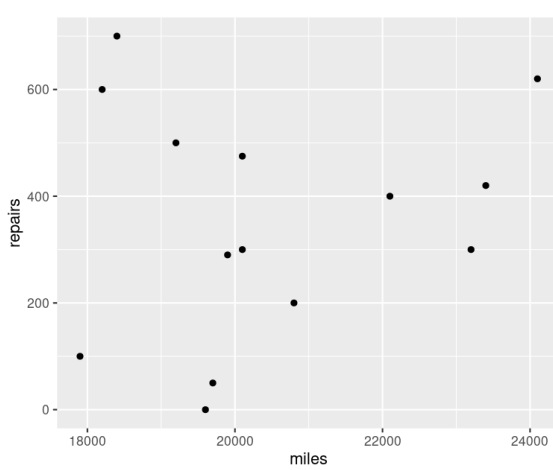


We can see a general trend: the **repairs** are high when the number of **oil changes** is low, and the **repairs** are low when the number of **oil changes** is high. That's a very clear pattern.

Now let's explore the other predictor- **miles driven** – in relation to **repair costs.** This time we will create the display with **ggplot:**

```
library(ggplot2)
ggplot(oil, aes(x=miles, y=repairs)) + geom_point()
```

The output is as follows:



Although this visualization looks great in ggplot, we don't see as clear a pattern as we did with **oil changes.** So just to start simple, lets build our first model with only **oilChanges** as the independent variable.

Here is the code for building a linear model:

```
model1 <- lm(formula = repairs ~ oilChanges, data = oil)
```

The **lm** command creates a data structure that contains everything we need to know about the results. We want to hang on to all that information, so we store the output of **lm** in an object called **model1.**

Note that the **lm** command takes two parameters:

1. The first tells **lm** to use **oilChanges** to predict **repairs**. The squiggly line [~] which is called a **tilde** character, tells **lm** which independent and dependent variables are to be modeled, with the entry on the left being the dependent variable and everything on the right being the predictors.

2. The other parameter tells **lm** the name of the daraframe where we have stored the training data. Note that because we supplied the name of the dataframe, we do not need to use the $ notation with the predictors (i.e., no need for **oil$repairs** or **oil$oilChanges**).

In the next command, we request an overview of the contents of our model:

```
summary(model1)
```

The output is as follows:

```
Call:
lm(formula = repairs ~ oilChanges, data = oil)

Residuals:
     Min       1Q   Median       3Q      Max 
-136.208  -48.195   -0.211   54.782  119.803 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  652.191     40.537  16.089 1.74e-09 ***
oilChanges   -71.994      8.202  -8.778 1.44e-06 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 82.72 on 12 degrees of freedom
Multiple R-squared:  0.8653,	Adjusted R-squared:  0.854 
F-statistic: 77.05 on 1 and 12 DF,  p-value: 1.436e-06
```

Whoa! That's a lot of information that R has provided for us, and we need to use this information to decide whether we are happy with this model.

Being happy with the model involves several different considerations and there is no simple rule. To start, we will look at the **R-squared value**, also known as the **coefficient of determination**.

The **R-squared value** can be interpreted as the **proportion** of the **variation in the dependent variable** that is accounted for by the **whole set of independent variables** (in this case just one independent variable).

1. An R-squared value of 1.0 would mean that the X variables made perfect predictions of the Y variable— every point in the training data fell right on the line.

2. An R-squared value of zero would indicate that the X variables did not predict the Y variable at all.

3. R-squared cannot be negative.

In the output, you can see two different R-squared values. You are always safe if you focus on the **adjusted R-squared value**. The adjusted R-squared value considers the number of **predictors** and the **sample size**.

- With a small sample and many predictors, adjusted R-squared will be much smaller (and more correct!) than plain R-squared.

- With a larger sample and not too many predictors, adjusted R-squared and plain R-squared will be about the same.

In this example, there was an adjusted R-squared of 0.854, which means that the **oilChanges** variable accounts for about 85.4% of the **repair** costs (stated another way, **oilChanges** can explain 85.4% of the variation in **repair** costs).

Although the adjusted R-squared is perhaps the most important output to evaluate from the model, let's explore some of the other outputs.

- First, at the top of the output we can see the actual **call** used to create the model. This is helpful if we are exploring a model but couldn't review the actual R code used to create the model.

- Next, we can see the **residuals,** which are the differences between the actual observed values and the predicted values. These residuals are also known as the **errors of prediction.** We see five summary numbers concerning residuals. One way to explore how well the model fits the data is to see whether the error (residual) is symmetrically distributed across these five summary points and for the median to be close to zero.

- The next section in the output describes the **coefficients** of the model. The coefficients are two constants that represent the intercept and slope terms in the linear model (remember the equation Y = bX + a: the slope is b, and the intercept is a.

In this example, the coefficient estimate contains two rows: the first row is the **intercept**. The intercept, in our example, is essentially the expected value of the **repairs** when there were zero **oilChanges**. The second row in the coefficients is the slope, or in our example, the effect that **oilChanges** have on vehicle **repairs.**

The **slope** in our model is saying that for every additional oil change, the expected repairs go down by $71.99. It might help if you ask yourself, "A one unit change in X leads to how much change in Y?" In this case an increase in X of 1 leads to a change in Y of -71.99.

- Finally, one other output to explore is the **p-value**. The **Pr(>|t|)** abbreviation, which is typically known as a p-value, describes the probability of observing any value equal or larger than the absolute value of a t-statistic under the assumption that the X and the Y are unrelated.

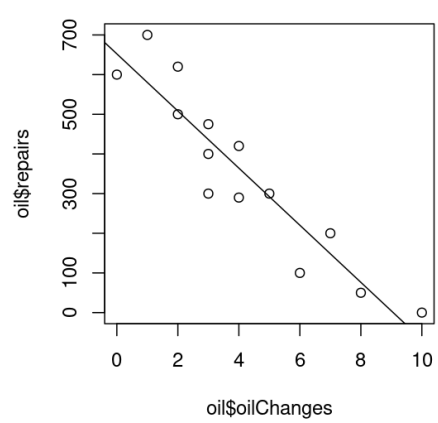
A small p-value is preferred: it indicates that it is unlikely to have observed a slope value this large because of sampling error. In most statistical analyses, a p-value under 0.05 is considered a good cutoff point.

In our model example, the p-values are very close to zero. Note **the "signif."** codes associated to each estimate. The asterisks are a shorthand for different cutoffs for p-values. If we see at least one asterisk, we can conclude that a systematic relationship probably exists between **oilChanges** and **repairs.**

Note that we can show the line of best fit (based on the model) to the X-Y plot of **repair costs** against **oilChanges** with a call to abline (pronounced A-B line) that includes the output of the **lm** command as its only argument.

```
plot(oil$oilChanges, oil$repairs)
abline(model1)
```

The output is as follows:



The previous model suggests that we should as many oil changes as possible. For example, the line predicts very low (almost zero) repairs if we do nine or more oil changes, but about $680 if we do no oil changes.

Now, lets try using both **oilChanges** and **miles** to create a new linear model. Perhaps both variables could do a better job predicting repair costs (as opposed to just using **oilChanges**). We can do this with the following R code, which includes the two predictors by putting a plus sign between them:

```
m <- lm(formula = repairs ~ oilChanges + miles, data=oil)
summary(m)
```

The output is as follows:

```
Call:
lm(formula = repairs ~ oilChanges + miles, data = oil)

Residuals:
     Min       1Q   Median       3Q      Max 
-130.488  -53.810   -1.712   46.301  151.182 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 343.26567  231.42285   1.483    0.166    
oilChanges  -71.98591    7.93052  -9.077 1.93e-06 ***
miles         0.01508    0.01114   1.354    0.203    
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 79.98 on 11 degrees of freedom
Multiple R-squared:  0.8845,	Adjusted R-squared:  0.8635 
F-statistic: 42.12 on 2 and 11 DF,  p-value: 6.982e-06
```

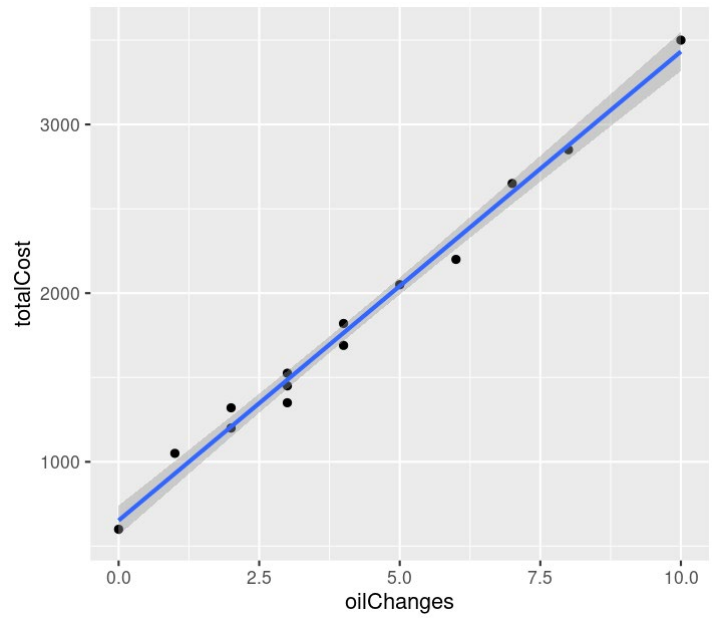
We can see that the new model is slightly better (a slightly higher adjusted R-squared value of 0.8635 compared to the previous model's 0.854). You can also see that **oilChanges** has *** but miles does not. This is because the model calculated that only the slope on **oilChanges** was statistically significant. This result is not surprising, because our initial X-Y plot suggested that miles did not show an obvious pattern in relation to **repair costs**. So, in this case, we could stick with our original model where we predict **repair costs** as a function of the number of oil changes.

In the work we have done so far, we've ignored one important element of the situation: we did not factor the cost of an oil change into our analysis. Let's assume that oil changes are very expensive. After all, when the car is having its oil changed, the driver cannot use the car for company business. We also must pay the mechanic and buy the oil and the filter. Putting that all together, let's model the cost of the oil change as $350 per oil change.

The following code creates additional columns in our dataset to compute and show the total cost for each car (which is the cost of the repairs plus the cost of doing the oil changes. The code then plots the results (this time using ggplot):

```
oil$oilChangeCost <- oil$oilChanges * 350
oil$totalCost <- oil$oilChangeCost + oil$repairs
ggplot(oil, aes(x=oilChanges, y=totalCost)) +
  geom_point() + geom_smooth(method="lm")
```

The output is as follows:



Now the analysis shows that we shouldn't do any oil changes at all because the lowest total cost appears at zero oil changes. Notice that when using **ggplot,** we use the "**geom_smooth**" command, which is similar to the **abline( ) function** in the core R package. The gray area around the blue line indicates the uncertainty around the prediction at any given level of X. We can see that the model has less uncertainty around five oil changes, as compared to zero or 10 oil changes. This makes sense because there were data points in and around those middle values. Let's double check our results by having R predict the total repair costs for a given number of oil changes with the following R code:

```
m <- lm(formula = totalCost ~ oilChanges, data=oil)
test = data.frame(oilChanges=0)
predict(m, test, type = "response")
```

The output is as follows:

```
652.191
```

```
test = data.frame(oilChanges=5)
predict(m, test, type="response")
```

The output is as follows:

```
2042.219
```

```
test = data.frame(oilChanges=10)
predict(m, test, type="response")
```

The output is as follows:

```
3432.247
```

Note that we did not review any summary statistics for this model such as R-squared. That's because our calculation of **totalCost** mathematically contains a direct influence from **oilChanges**. This artificially inflates the **R-squared** and the **p-values**.

We can see that as the number of oil changes increases, so does the predicted total cost of maintenance, Of course, we still may not have all the important information that should be modeled. For example, perhaps the resale value of each car after it has completed its three years of service was also affected by maintenance. This represents a situation that often arises when doing data science. That is, we have some data to create a model, but we are not sure whether the model covers all the circumstances that could possibly impact the results of our analysis. We may be missing certain variables that we need to do a better job of modeling, or some of the data we have might be contaminated by influences that we cannot see. That's why it is important, when reporting the results of a predictive model, to be cautious about claiming that x causes y or that increasing x is guaranteed to decrease y or the other way around. The statistics can provide helpful guidance, but accurately predicting the future is always quite tricky.

USING THE CARET PACKAGE

In addition to using the **lm** function directly, we can load and use the **Caret package**, which is an R package that supports many machine learning techniques. The Caret package (short for **Classification and Regression Training**) contains functions to simplify the model training process for both regression and classification. Caret does not actually do the modeling; it just provides a consistent interface for accessing many individual specialized R packages. For example, when you build a **linear model** using **Caret**, **Caret** calls the **lm** function, just as we did earlier in this chapter. This is important because when you try to run the R code, **Caret** might note additional packages that need to be installed.

Keeping this in mind, you can see below that we can use **Caret** to create a linear model, just as we used **lm** directly to generate a linear model earlier in the book. Note that **the trControl** argument refers to how we should control the training of the model, but we do not have to worry about that just yet, which is why we use method of "none".

```
#install and library the caret package as needed
install.packages("caret")
library(caret)

#create the model, letting caret's train function
#call the "lm fucntion

model <- train(repairs ~ oilChanges, data=oil,
               trControl=trainControl(method = "none"),
               method = "lm")
summary(model1)
```

The output is as follows:

```
Call:
lm(formula = repairs ~ oilChanges, data = oil)

Residuals:
     Min       1Q   Median       3Q      Max 
-136.208  -48.195   -0.211   54.782  119.803 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  652.191     40.537  16.089 1.74e-09 ***
oilChanges   -71.994      8.202  -8.778 1.44e-06 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 82.72 on 12 degrees of freedom
Multiple R-squared:  0.8653,	Adjusted R-squared:  0.854 
F-statistic: 77.05 on 1 and 12 DF,  p-value: 1.436e-06
```

As you can see, a summary of model1 shows that it is the same as the model that was generated when directly using the lm function (i.e. in this situation, using train() or lm( ) directly will result in the same model).

Partitioning into Training and Cross Validation Datasets

Our predictive model, created from our ML algorithm, will inevitably make mistakes. For example, the model might predict rain for Thursday, but it turns out that Thursday is sunny. For any prediction problem, we want our error value to be as small as possible. Improving prediction accuracy (i.e., reducing the error rate) is the basic goal of ML.

Thus, we use a substantial number of training cases to help the algorithm discover the underlying pattern. Once the algorithm has detected the pattern from the training data, it stores a set of numbers and other information that describe the trained ML model. We can then apply the model to some new X data (i.e., new values of our predictors.)

Here's a new trick: after we find out what really happens with that new data, we can see how well our model performed on new data by checking the predicted values against the actual events.

For example: We can use our new model to suggest the number of oil changes (or to predict whether tomorrow will be sunny). Rather than waiting for these future events to happen (and then seeing the accuracy of the model), we can just divide the data that we already have into two chunks, a **training dataset**, and a **test dataset** (which is also sometimes called a holdout sample or a **cross validation sample**). For example, if we do not want to wait a couple of years to see the accuracy of our maintenance model, we might randomly sample two-thirds of that data that we already have to train our model and save the rest of the data to use in testing our trained model.

Let's redo our car maintenance example, but this time, have a dataset to train (i.e., build) the model and a different dataset to test (i.e., evaluate) the model. There is no universal approach on how much of the data should be set aside for training, and how much for testing, but as a rule of thumb, when using this partitioning approach, you can use somewhere between two-thirds to three-quarters of the dataset to train and the remainder to test. Furthermore, when using the partitioning approach, it is important to randomize the selection of cases for the training and test sets to ensure that there is no systematic bias in the selection of cases.

Fortunately, the **Caret package** contains the **createDataPartition** command, which randomly samples without replacement from a list of numbers ranging from one to the final element index of the dataset to be partitioned, which we can use to create our train and test datasets. One important aspect of the **createDataPartition** command is that the command uses the dependent variable (the outcome that we hope to predict) as an input. This input variable helps the function to ensure that the differing values of Y are equally well represented in both the training and test sets.

Let's think through an **example** with a small number of cases: **February weather in Syracuse, New York**. Our overall dataset contains 28 days, of which it was sunny on 16 days and cloudy (and snowed) on 12 days —that's about 57% sunny days. We tell **createDataPartition( )** that we want half of our data (14 days) for training and the remainder (14 days) as a hold out for testing. The **createDataPartition** command will randomly sample to make sure that both datasets contain a random selection of eight sunny days and six snowy days- again about 57% sunny days in each dataset. Let's try it with some R code to see what happens:

```
#use the set.seed to enable repeatable results
set.seed(10)

#make sure caret is loaded and ready
library(caret)

#create our weather outcome variable
sunnyDays.percentCloudy <- sample(1:10, 16, replace=TRUE)
cloudyDays.percentCloudy <- sample(90:100, 12, replace=TRUE)
percentCloudyAllDays <- c(sunnyDays.percentCloudy, cloudyDays.percentCloudy)

#The p=0.5 splits the dataset in half
trainList <- createDataPartition(percentCloudyAllDays, p=0.5, list=FALSE)

#Show the results
data.frame(trainList, cloudPercent = percentCloudyAllDays[trainList] )
```

The output is as follows:

```
   Resample1 cloudPercent
1          2           10
2          3            7
3          4            8
4          5            6
5          8            8
6          9           10
7         12            2
8         16            6
9         17           96
10        18          100
11        21           94
12        22           98
13        23           91
14        27           90
15        28           96
```

The middle column of output above, under the heading "**Resample1**," shows the row indices (from the original data) that **createDataPartition( )** randomly sampled. You can count the values in the right-hand column to see that we have eight sunny days and six snowy days (57% sunny), the same as the full dataset. Also remember by putting a minus sign in front of trainIndex, you can get all of the rows that belong in the test dataset instead. Try it! Because this is a random set of indices, if the code ran again, without using the **set.seed(10)**, we would get different index numbers in the **trainList.**

```
#create train and test datasets
trainData <- percentCloudyAllDays[trainList]
testData <- percentCloudyAllDays[-trainList]
```

Now we are ready to partition our oil dataframe dataset, remembering that the repairs attribute is our dependent variable, and so, we supply that attribute to the **createDataPartition** function.

```
#makes the sampling predictable
set.seed(123)

#Randomly sample for training dataset elements
trainList <- createDataPartition(y=oil$repairs, p=0.66, list=FALSE)
```

Note that we first use the **set.seed( )** function, so that when we rerun the code, we get the same results, and so that you can reproduce the same results as long as you use the **set.seed( )** function with the same value of 123. If you provide a different number, R will generate a different set of random samples.

We can confirm that the indices are a random subset of all the indices by looking at the first few cases:

```
head(trainList[,1])
```

The output is as follows:

```
> head(trainList[,1])
[1] 1 2 3 5 6 7
```

Now we are ready to generate our training and testing datasets from the original oil dataset by using the randomized set of indices that we created in the train List. As previously mentioned, because the test dataset is composed of all the datapoints that were not in the **train dataset,** we can use the negative sign with the **trainList** to create the testData variable:

```
#create train and test datasets
trainData <- oil[trainList,]
testData <- oil[-trainList,]
```

We now have two separate datasets, representing the **training** and **test** breakdown of the original data, and can use our training dataset to build our linear model. The following command generates a model based on the training dataset:

```
lm.model <- train(repairs ~ oilChanges + miles, data= trainData,
                  trControl=trainControl(method = "none"), method = "lm")

summary(lm.model)
```

The output is as follows:

```
Call:
lm(formula = .outcome ~ ., data = dat)

Residuals:
    Min      1Q  Median      3Q     Max 
-150.11  -28.22   10.85   28.57  108.01 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 379.03499  269.92491   1.404 0.203033    
oilChanges  -84.68756   11.75110  -7.207 0.000176 ***
miles         0.01618    0.01265   1.279 0.241688    
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 82.05 on 7 degrees of freedom
Multiple R-squared:  0.8866,	Adjusted R-squared:  0.8542 
F-statistic: 27.37 on 2 and 7 DF,  p-value: 0.0004905
```

This code is like the code we ran previously, where we used both oil changes and miles to predict repairs. We can see that the adjusted R-squared is about the same as before. This wont always be the case: with a different random sample of observations in our training data, we might get a value of adjusted R-squared that came out slightly lower or slightly higher. Additionally, we should not be surprised to see that oilChanges is an important predictor, but that the miles attribute was not.

To understand the quality of our model in predicted novel data, we first use the **predict** function to generate the predicted values. Then, we can look at the squared correlation between the predicted values of Y and the original values for the Y variable (repairs) in the **test dataset**. This shows a value roughly the same as the R-squared value from the training data. Again here, we might see a somewhat lower or high value if the random sampling that partitioned the training and test datasets turned out differently.

```
predictValues <- predict(lm.model, newdata = testData)
cor(predictValues, testData$repairs)^2
```

The output is as follows:

```
[1] 0.9724484
```

USING K-FOLD CROSS VALIDATION

Partitioning your data into train and test datasets can work reasonably well if you have a lot of data, let's say more than 1,000 observations/rows. However, if you do not have a lot of data, or if your data is very complex, then partitioning your data might not be the best strategy.

In stratified k-fold strategy, you tell your training algorithm to follow these four steps:

1. Randomly divide all your available data into k-separate chunks (it turns out that k = 10 is often the best choice).

2. Let the algorithm train a model where one of the k-separate chunks is kept out of the training data and is rather used as the test dataset for the model.

3. Built with the other k-1 chunks.

4. Combine the results of the k separate models into one overall model; and report the error rate and accuracy of the overall model.

Luckily, it is easy to use **k-fold cross validation** using the **Caret package**. So, let's try it using the same **train** and **test** dataset from our car maintenance example. As you can see in the following code, to use **k-fold cross validation** within the **Caret package**, we first call the **trainControl** function, stating that we want to use repeated cross validation "**repeat-edcv**," and defining k (the number of folds). The "repeated" part of the cross validation is important in doing k-fold cross validation, as it generates different results that depend on how we split the data. So, to account for this variation, one typically repeats the whole **k-fold cross validation** multiple times, which are then averaged together. We then pass the results of that train control function to the train function.

```
trctrl <- trainControl(method="repeatedcv", number=10, repeats=3)
lm.model1 <- train(repairs ~ oilChanges + miles,
                   data = trainData,
                   method = "lm",
                   trControl = trctrl)
```

We get the following:

```
Warning message:
In nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,  :
  There were missing values in resampled performance measures.
```

One of the things that you may notice from the code is that the **train function** does most of the work of **k-fold cross validation** for you. You can just specify the number of folds that you want- again 10 is often a good choice, and the number of times the cross validation should be repeated, and then let the algorithm do its thing.

So now, with our specified parameters, the **lm** algorithm will run 30 times! That is, R will do **10-fold cross validations** three times (with each 10-fold cross validation having 10 iterations being repeated three times). Note that each fold (iteration) will be done with 90% of the training data, and each fold will be evaluated (validated) with 10% of the training data. Don't worry if you get a warning when running this code. The warning is because we are working with such a small dataset. Let's look at the model generated:

```
summary(lm.model)
```

The output is as follows:

```
Call:
lm(formula = .outcome ~ ., data = dat)

Residuals:
    Min      1Q  Median      3Q     Max 
-150.11  -28.22   10.85   28.57  108.01 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 379.03499  269.92491   1.404 0.203033    
oilChanges  -84.68756   11.75110  -7.207 0.000176 ***
miles         0.01618    0.01265   1.279 0.241688    
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 82.05 on 7 degrees of freedom
Multiple R-squared:  0.8866,	Adjusted R-squared:  0.8542 
F-statistic: 27.37 on 2 and 7 DF,  p-value: 0.0004905
```

You can see that the adjusted R-squared was the same as when using the full dataset. Just as before, we can predict unknown (i.e., **testData**) values and calculate the squared correlation between those predictions and the true values from **testData.**

```
predictValues <- predict(lm.model1, newdata=testData)
cor(predictValues, testData$repairs)^2
```

The output is as follows:

```
[1] 0.9724484
```

You might wonder why using the **k-fold repeated cross validation** didn't improve our results. This is because a linear model is a simple model, not prone to overfitting, and hence, using **k-fold cross validation** did not improve our resulting model. However, as we move to more advanced models, overfitting becomes an increasing concern and k-fold cross validation becomes increasingly useful.

A related question you might have is that since there is some **validation data** for each iteration when using **k-fold cross validation**, which can provide an accuracy/error estimate, why can't we use all the data for the creation of the model (rather than holding out some test data). The answer is: It is generally best to use all our data for training assuming we are using **k-fold cross validation** and just forget about having a test set at all. The **k-fold cross validation** process helps us avoid overfitting and we can feel assured that the resulting model quality will hold up in future samples.

Important R functions

- **abline( ):** Plots a best-fitting line on the top of a scatterplot.
- **cor( ):** Pearson's correlations among variables.
- **createDataPartition( ):** Chooses elements at random from a vector.
- **filter( ):** Tidyverse function to select a subset of rows.
- **ggplot( ):** Uses geom_point and stat_smooth.
- **lm( ):** Stands for linear models and, for this chapter, multiple regression.
- **plot( ):** Is a general purpose graphing function that has many uses in R.
- **predict( ):** Uses a model to predict a variable (output).
- **sample( ):** Chooses a sample of elements from an object.
- **sample_n( ):** Tidyverse function to sample a set of rows from a table.
- **ste.seed( ):** Controls randomization for tasks like sampling.
- **summary( ):** Provides a statistical summary of numeric data.
- **train( ):** Uses the caret package to train a model
- **trainControl( ):** Uses the caret to set up control parameters for training a model.
- **View( ):** Shows a dataframe in an easy-to-read format.